Chapter 3

Database Construction and Data Preprocessing

The news database provides the starting point for the empirical analysis in this thesis. It contains articles published between 2014 and 2023 and combines material from five newspapers: *Financial Times (FTXX), Wall Street Journal (WSJX), New York Times (NYTX), Los Angeles Times (LATX) and USA Today (USAT).*

The raw files are initially stored in a staging table to make it possible to check their content before they enter the main database. The checks concern basic information such as the publication date, newspaper source, language, title, and article body. Records that pass these checks are stored in a common format.

Duplicates require a separate treatment. Duplicate texts can occur more than once in the original data, but repetition is not always exact. The database therefore distinguishes exact duplicates from near-duplicates and identifies them using two different methods.

These choices matter for the empirical analysis. If the same article appears several times, for example, it receives more weight than it should when news coverage is measured. Errors in dates or source information can create similar problems when articles are compared across newspapers or over time. The cleaned corpus used in the following chapters is constructed after these issues have been addressed.

3.1 Dataset and Database Architecture

3.1.1 News Corpus

The unit of observation is an individual news article. The source files contain both textual information and article-level metadata: the article title and body, publication date, language, source, an original sentiment variable, a duplicate indicator, a data-type field, and the article URL.
The staging table contains 2,374,643 records. After validation and transfer to the main table, `news_articles` contains 2,373,999 records. Deduplication is performed subsequently on this set of articles.

3.1.2 Database Architecture

The news corpus is stored in PostgreSQL, in the `research` database. A dedicated schema, `risk_emergence`, contains the tables and views developed for the project. Its structure keeps the original ingestion layer separate from the data and variables produced later in the analysis.

Two tables are directly involved in the initial preparation of the news corpus. The first is `news_articles_staging`, where records from the source files are initially stored. The second is `news_articles`, which is the main article table used by the rest of the database.

The staging table has a deliberately simple structure. Its fields are stored as text and restrictive constraints are avoided at this stage. Records can therefore be loaded before dates, sources, and other fields are checked and converted to their appropriate types. The table also stores `_raw_line_number` and `_source_file`. These fields identify the original row and file associated with each record, allowing problematic observations to be traced back to the source data.

Once the required checks and conversions have been performed, the records are transferred to `news_articles`. This table retains the original article information and adds fields created during data preparation. Among them are `word_count`, `char_count`, language and duplicate flags, and `content_hash`. The latter is derived from the article body and is later used to identify exact duplicates. The table also contains full-text representations of the title and body, stored in `title_tsv` and `body_tsv`.

The database extends beyond these two tables. Other objects store newspaper information, theme scores, article embeddings, firm mentions, and aggregated indicators. They are linked to `news_articles`, which acts as the central table for the news component of the project.

3.2 Data Ingestion and Validation

3.2.1 Raw Data Ingestion

The original news data are stored in UTF-8 encoded CSV files using the vertical bar (`|`) as delimiter. The ingestion procedure is implemented in Python and proceeds through the staging table before records are transferred to the final database.

Before importing the observations, the expected CSV header is checked against the nine fields: Title, Body, Date, Language, Source, Sentiment, IsDuplicate, dataType, Url. Each raw observation is required to contain exactly nine fields. Rows that do not satisfy this structural requirement are not transferred as valid observations and are recorded as malformed rows.

The ingestion is performed in batches rather than by loading the entire corpus into memory. During this process, the original row number and source file are appended to each record through the variables `_raw_line_number` and `_source_file`. The resulting observations are inserted into `risk_emergence.news_articles_staging`.

This staging-first design separates data acquisition from data transformation. In particular, observations can be inspected and validated before type conversions and normalisation rules are applied.

3.2.2 Staging Validation

Once the raw observations have been imported, the staging table is subjected to a series of validation checks. Given the size of the corpus, the validation is performed incrementally using a server-side cursor rather than loading all observations simultaneously into memory.
The validation procedure examines the main variables and it verifies that:

- publication dates can be parsed into an accepted date format;
- the newspaper source can be mapped to a recognised source;
- the original sentiment value is either missing or numerically interpretable;
- the original duplicate indicator can be interpreted as a Boolean value;
- the article title is not empty;
- the article body is not empty;
- the language value can be classified consistently.

Multiple date formats are accepted during parsing in order to accommodate differences in the raw source files. The validation stage therefore separates the question of whether a date can be interpreted from the subsequent normalisation of that date into a common database representation.
Similarly, the sentiment variable is validated only as an original metadata field. No new sentiment analysis is performed during this stage. Missing sentiment values are allowed, while non-missing observations must be convertible to a numerical representation.
The original duplicate indicator is also checked independently from the deduplication procedure described later in this chapter. This distinction is important because the deduplication performed for the thesis does not rely exclusively on the duplicate information supplied in the raw files.
After transfer to the final database, quality-control queries confirmed the presence of 2,373,999 observations and no missing values in the key fields `title`, `body`, `publication_date`, and source information. The observed date range corresponds to the intended sample period, from January 1, 2014 to December 31, 2023.

3.3 Data Normalisation

After validation, the observations are transformed into the format required by the final database. The normalisation procedure includes date parsing, source mapping, language normalisation, data-type conversion, and the construction of variables required by the subsequent analytical pipeline.

In particular: the original source codes are mapped to the corresponding newspaper identifiers in the database, the language information is converted into a consistent representation that can subsequently be used to identify English-language observations and the publication dates are converted into

a common database date representation.

The final table also contains measures that describe the textual content of each article. These variables support both data-quality inspection and later text-processing operations. An additional variable, `content_hash`, is constructed from the article body using the SHA-256 hashing algorithm. For an article with textual body B_i, the hash can be represented as

$$h_i = \text{SHA256}(B_i). \tag{3.1}$$

The content hash provides a deterministic representation of the article body and is the basis for the exact-deduplication procedure described in Section 3.4.

3.4 Deduplication

Duplicate news articles are another possible cause of measurement error. Multiple copies of an article would artificially inflate the counts of particular themes, thus impacting the metrics of coverage, novelty, and temporal variation. For this reason, deduplication is carried out prior to the use of the corpus in the following NLP and embedding processes.
One measure of deduplication is not enough for this task. Identical articles can be determined using deterministic means, but articles that are different due to slight editorial adjustments are to be identified through the process of comparing similarities. The deduplication process includes two steps:

$$\text{Exact deduplication} \quad \longrightarrow \quad \text{Near-duplicate detection.} \tag{3.2}$$

3.4.1 Exact Duplicate Detection

Exact duplicates are identified using the SHA-256 hash of the article body introduced in Equation 3.1. Two observations with the same content hash are treated as copies of the same textual content.
Only one article from each group of articles sharing the same `content_hash` is kept. The articles are ordered by `article_id`, and the observation with the smallest identifier is retained. All remaining observations in the group are flagged as exact duplicate.

Conceptually, if G_h denotes the set of articles sharing hash h, the retained article is

$$i_h^* = \arg\min_{i \in G_h} \texttt{article_id}_i. \tag{3.3}$$

All other observations belonging to G_h are flagged as exact duplicates.

Results This procedure identified 70,964 duplicate so, starting from 2,373,999 articles, the exact-deduplication stage therefore left 2,303,035 articles eligible for the subsequent near-duplicate analysis.
It is important to note that duplicate observations are not deleted from the database. The `is_duplicate_flag` records their status while preserving the original records. This allows the complete ingestion history to be retained and makes the deduplication procedure auditable.

3.4.2 Near-Duplicate Detection

Exact hashing cannot identify articles that contain the same content but differ slightly in the text. Examples include limited insertions or deletions, minor editorial changes, or alternative versions of essentially the same article. A second deduplication stage was therefore implemented using MinHash and Locality Sensitive Hashing (LSH).

Only the 2,303,035 observations not already classified as exact duplicates enter this stage.

Each article body is represented as a set of consecutive five-word shingles. If the tokenised body of an article is

$$(w_1, w_2, \ldots, w_n), \tag{3.4}$$

its shingle representation is

$$S_i = \left\{(w_j, w_{j+1}, w_{j+2}, w_{j+3}, w_{j+4})\right\}_{j=1}^{n-4}. \tag{3.5}$$

MinHash signatures are then computed using 128 permutations. The signatures provide an efficient approximation of the similarity between the shingle sets and are indexed using MinHash LSH with a threshold of 0.90.

LSH approximate output is not used directly to classify an observation as a duplicate. Instead, candidate pairs identified through LSH are subjected to an exact Jaccard-similarity calculation.

For two articles represented by shingle sets A and B, Jaccard similarity is defined as

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}. \tag{3.6}$$

A candidate is classified as a near-duplicate only when

$$J(A, B) \geq 0.90. \tag{3.7}$$

The high similarity threshold makes the procedure deliberately conservative. Its purpose is to identify copies or near-copies of the same textual content rather than articles that merely discuss the same event or topic. In particular, independently written articles from different newspapers covering the same event should not be classified as duplicates solely because they are semantically related.

Results The LSH stage generated 59,903 candidate matches, of which 59,418 were subjected to exact Jaccard comparisons. The final procedure identified 53,780 additional near-duplicate articles satisfying the 0.90 Jaccard threshold.

3.4.3 Construction of the Final Analytical Corpus

After both deduplication stages, the number of articles available for the subsequent empirical analysis is

$$N_{\text{valid}} = N_{\text{total}} - N_{\text{exact}} - N_{\text{near}} \tag{3.8}$$
$$= 2,373,999 - 70,964 - 53,780 \tag{3.9}$$
$$= 2,249,255. \tag{3.10}$$

This corresponds to approximately 5.25% of the corpus.

Observations were not excluded in any way using the separate `is_excluded` flag at the end of this preprocessing step. The difference between the normalised database and the analytical corpus can thus be explained by only the two deduplication techniques mentioned above.
The resultant corpus consists of 2,249,255 articles and forms the document universe on which the following stages of the analysis rely. Practically, the observations are made available via the `risk_emergence.v_valid_articles` view, offering a unified input for the following stages of thematic scoring and document representation.
This approach allows preserving both the initial database and the process of the construction of the analytical sample. The exact duplicates, near-duplicates, and any other exclusions are encoded as separate fields of the database, rather than being deleted from the dataset.